Swadeshi Kitchen

SYSTEM ARCHITECTURE & CUSTOMIZATION MANUAL

1. Executive Project Summary

This application is a modern, high-performance web platform designed specifically for cloud kitchens. It provides customers with an interactive menu, responsive cart checkout, and real-time order tracking. Concurrently, it equips the business owner with a robust administrative dashboard to configure and oversee daily operations.

The system is architected as a React Single Page Application (SPA) utilizing Vite, TypeScript, Tailwind CSS, and Framer Motion. By utilizing a hybrid storage engine, the system remains 100% functional out-of-the-box locally (via client browser state) and can be upgraded instantly to support cloud-wide database synchronization using Google Firebase.

2. System Architecture & Dashboard Connection

The core interaction of the platform is the bridge between the **Admin Dashboard** (operations) and the **Customer Website** (consumers). Below is an explanation of how they connect and communicate:

A. The Shared Storage Engine

Both panels run inside the same React context but render different views depending on the routing path. They query a single data access layer defined in `src/lib/store.ts` and `src/lib/firebase.ts`. This layer abstracts reading and writing from local storage or cloud databases.

B. Dynamic Timing, Menu, and Testimonials Sync

- When the administrator adds a new dish, edits a price, disables a discount code, or modifies the opening hours (e.g. changing opening time to `8:00 AM`) in the Admin Panel, the dashboard saves this state to the database.
- The Customer Page loads this data inside a React `useEffect` hook on mount. As a result, any menu updates, prices, or timing changes appear instantly to the customers.

C. Real-time Status Tracking Flow

The live order status bar (Order placed ➔ Cooking ➔ Packed ➔ Out for delivery ➔ Delivered) is powered by a dynamic status polling pipeline:

1. **Order Placement:** When a customer submits their cart, the system creates a unique order record with status `Pending` and forwards the invoice details to the kitchen via WhatsApp.
2. **Dashboard Tracking:** The order immediately appears inside the Admin Panel's **Orders** list.
3. **Status Changes:** The administrator changes the order status using a dropdown selector. This writes the new status (e.g. `Packed`) to the database.
4. **Customer Status Polling:** The customer-facing tracking page runs an active polling loop using a JavaScript interval. Every 3 seconds, it queries the database for the active order ID. When a status change is detected, the UI updates automatically to light up the progress slider!

3. Repository Directory Structure

Directory / File	Purpose	Key Customization Action
`vite.config.ts`	Vite packaging configuration.	Set the `base` folder for GitHub Pages hosting.
`src/App.tsx`	Application router & entry.	Switch between `BrowserRouter` and `HashRouter`.
`src/Home.tsx`	Main customer portal landing.	Change layout columns, sections, scrolling effects, and WhatsApp template text.
`src/lib/store.ts`	Data layer / State store.	Set default menu dishes, categories, pricing grids, and brand logos.
`src/lib/firebase.ts`	Cloud database interface.	Add Firebase credentials and handle read/write functions.
`src/admin/Login.tsx`	Dashboard login controller.	Define permitted email addresses and password criteria.
`public/404.html`	GitHub Pages router helper.	Direct URL requests to HashRouter equivalent pages.

4. How to Duplicate this Website for a New Brand

To copy this codebase and set it up for a new client or kitchen brand, follow these step-by-step instructions:

Step 1: Clone the Codebase & Create a New GitHub Repo

Clone this folder locally, remove the existing Git history (optional), create an empty repository on GitHub under your account, and push the source code there.

Step 2: Update the Hosting Base URL

Ensure the base asset path matches your new GitHub repository name. Open the config file and edit the `base` property:

vite.config.ts

```
export default defineConfig({
  base: '/your-new-repo-name/', // Must match the repository name exactly
  server: { ... }
```

Step 3: Modify Default Configurations

Set up the default branding, contact email, phone number, address, and default dishes for the new brand:

src/lib/store.ts

```
const defaultSettings: GlobalSettings = {
  hero: {
    title: 'Brand Title Here',
    subtitle: 'Brand Subtitle Details...',
    backgroundImage: 'URL to your hero image'
  },
  contact: {
    email: 'contact@brand.com',
    phone: '+919999999999', // Default phone number
    address: 'Kitchen Address, India',
    openingHours: 'Mon-Sun: 8:00 AM - 11:00 PM'
  },
  brand: {
    name: 'Brand Name',
    logo: '/your-new-repo-name/logo.png' // Ensure path matches new base
  }
};
```

Step 4: Update the Admin Credentials

Set the email address and password required to access the owner's dashboard:

src/admin/Login.tsx

```
if (email === 'owner@newbrand.com' && password === 'securepassword123') {
  localStorage.setItem('swadeshi-admin-auth', 'true');
  navigate('/admin');
}
```

Step 5: Customize Style Themes & Accent Colors

This website uses Tailwind CSS class design. Colors are managed primarily through utility classes:

1. To change the site accents (e.g. from Orange/Green to Blue/Indigo), open `src/Home.tsx` and files inside the `src/admin/` directory.
2. Perform a global search-and-replace to change color names. For example, replace `orange-600` with `blue-600` and `orange-700` with `blue-700`.
3. Customize base elements like borders and buttons inside the CSS file:

src/index.css

```
.btn-primary {
  @apply bg-orange-600 hover:bg-orange-700 text-white transition-all active:scale-95;
}
```

5. Deployment Commands

Use these command lines inside the project terminal to manage local development, build, and deploy the new website:

```
# 1. Install all dependencies
npm install

# 2. Run local development server
npm run dev

# 3. Test build compilation for production
npm run build

# 4. Save and commit changes to Git
git add .
git commit -m "feat: customize brand and configurations"
git push origin main
```

```
# 5. Build and Deploy live to GitHub Pages
npm run deploy
```

💡 GitHub Pages Hosting Activation Note

After pushing the build using `npm run deploy`, log in to your GitHub account web panel, go to **Settings ➔ Pages** on the repository, set the build source to **"Deploy from a branch"**, select the `gh-pages` branch, and click **Save**. The site will go live in less than 60 seconds.